DATA 3464: Fundamentals of Data Processing

Missing and Weird Data

Charlotte Curtis

October 1, 2026

Topic overview

- Written test 1 info
- What to do with missing data
- Detecting and handling outliers

Resources used:

- Feature Engineering Chapter 8
- Hands on Machine Learning with Scikit-Learn and Tensorflow/PyTorch, Chapter 2. Available at MRU Library
- Scikit-learn user guide: Chapters 2 and 7

Written test 1 info

- 1 **handwritten** reference page on 8.5x11" paper, single-sided
- A non-programmable emotional support calculator
- 6 questions MC, 9 TF, 3x long answer (similar to class practice)
- Total of 50 points (1 point per MC), 80 minutes
- Last semester's midterm is on D2L — it has concepts we haven't covered yet!
- Two tests worth 35% total — this one is 15% or 20%, whichever is best for you

Written test 1 content

What is *not* on the test:

- Clustering, because we skipped it
- Detailed knoweldge of gradient descent or probability distributions
- Any kind of Python syntax or coding

What *is* on the test?

- Concepts from lectures, labs, and assignments
- Up to and including basic imputation of missing values (today!)
- More emphasis on things you did in labs/assignments

The problem

- As you've seen, real-world data is messy
- Missing values are common, other values don't make sense
- We need to decide how to deal with these problems

> *What examples have we seen so far?*
>
> *Why might data be missing or weird*?*

Missing data

When data are missing in the *features* we have a few options:

0. Do nothing! Some algorithms (e.g. Decisions Trees) can handle missing values
1. Remove *features* with missing values
2. Remove *samples* with missing values
3. Invent a new value to represent "missingness"
4. **Impute** a value based on other data

> *Most important: understand why data are missing (more EDA!)*

Option 1: removing features

- If a feature has:
  - A high proportion of missing values, *and*
  - Little apparent relationship to the target *or*
  - its information is redundant with other features
- It may be reasonable to remove it entirely. You can drop it, e.g.:

```
df.drop(columns=['feature_name'], inplace=True)
```

  or (probably more reliable) just not select it when building your pipeline

Option 2: removing samples

- If:
  - Only a small number of samples have missing values
  - The values are missing completely at random

  Then you can drop them **from the training data**:

```
train_nona = train.dropna(subset=[["features","we","care","about"]])
```

- Good idea if the same samples have missing values from multiple features
- Still useful to explore *why* data are missing

> *What does this imply at inference time?*

Option 3: invent a new value

- Categorical features: add a new category for "missing"
- Add a new binary feature indicating whether the value was missing
- I have seen advice to use extreme values for numerical features, like the -1 income in the OKCupid dataset, but I'm not convinced this is a good idea

> *Case study: where missingness is informative*

Option 4: impute missing values

- Fill in the missing values with an "educated guess"
- Replace missing value with:
  - constant
  - mean, median, or mode (`most_frequent`)
- Use other features to infer missing value:
  - K-nearest neighbours
  - simple models to predict missing values
- Can be combined with option 3 to indicate missing features
- How much to impute? Feat.Engineering suggests no more than 20%

Choosing an imputation strategy

Strategy	When to use
Constant	When there is a reasonable default value
Mean	Numeric features with normal distribution
Median	Numeric features with extreme outliers
KNN	Relationship with other features
Missingness indicator	If missingness seems informative

Outliers

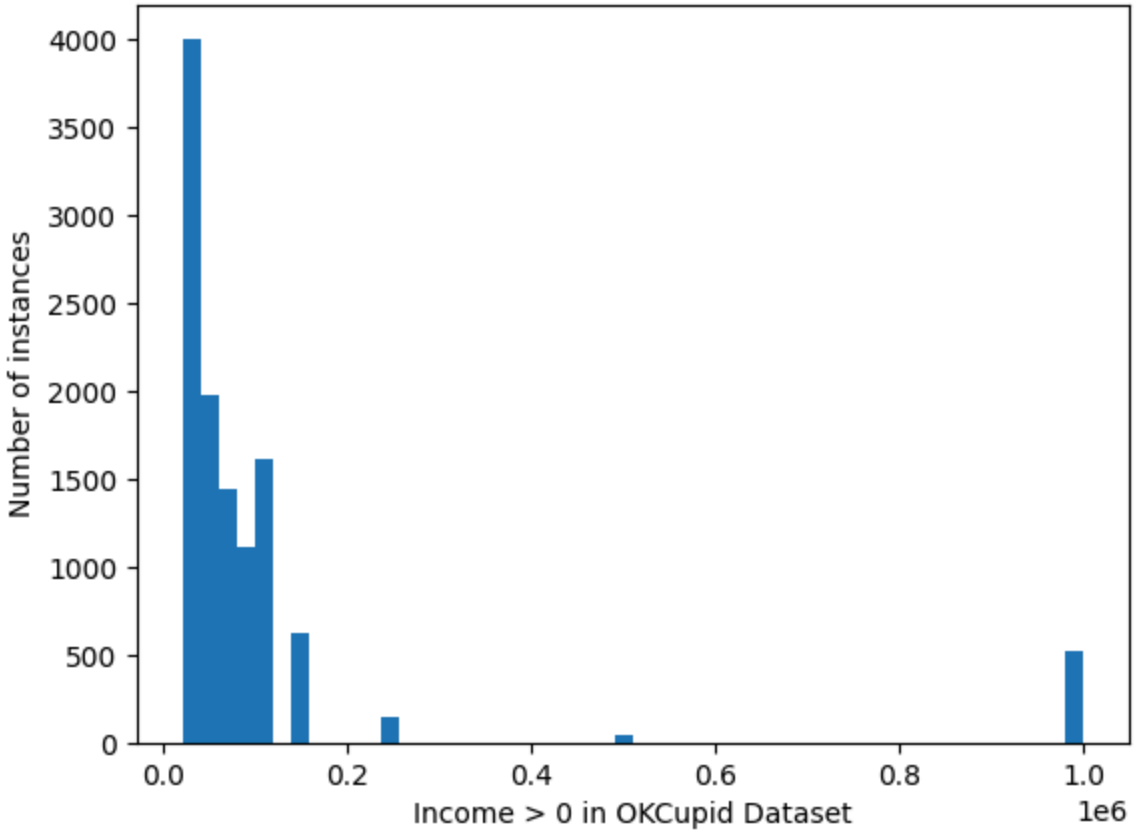
> *An outlier is an observation which deviates so much from the other observations as to arouse suspicions that it was generated by a different mechanism. — D. M. Hawkins*

- There are many entire books dedicated to outlier detection
- Useful for anomaly detection, e.g:
  - fraud detection
  - failure prediction
  - Many other applications
- Our focus is on dealing with outliers in preprocessing

Hawkins, D. M. (1980). Identification of Outliers.

Detecting outliers

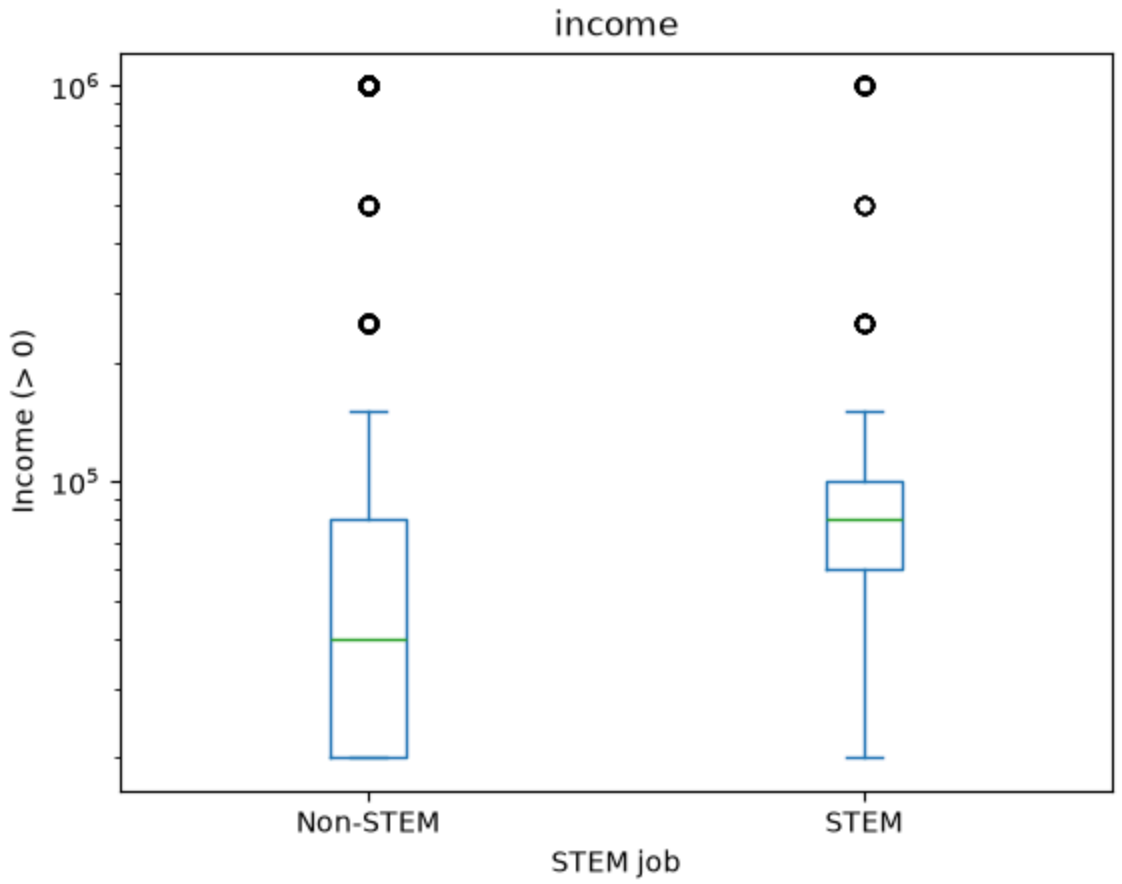
- Visually as part of EDA
- Statistically, e.g. $> 3\sigma$
- Algorithmically, e.g. Isolation Forests
- As usual, context and domain knowledge are essential



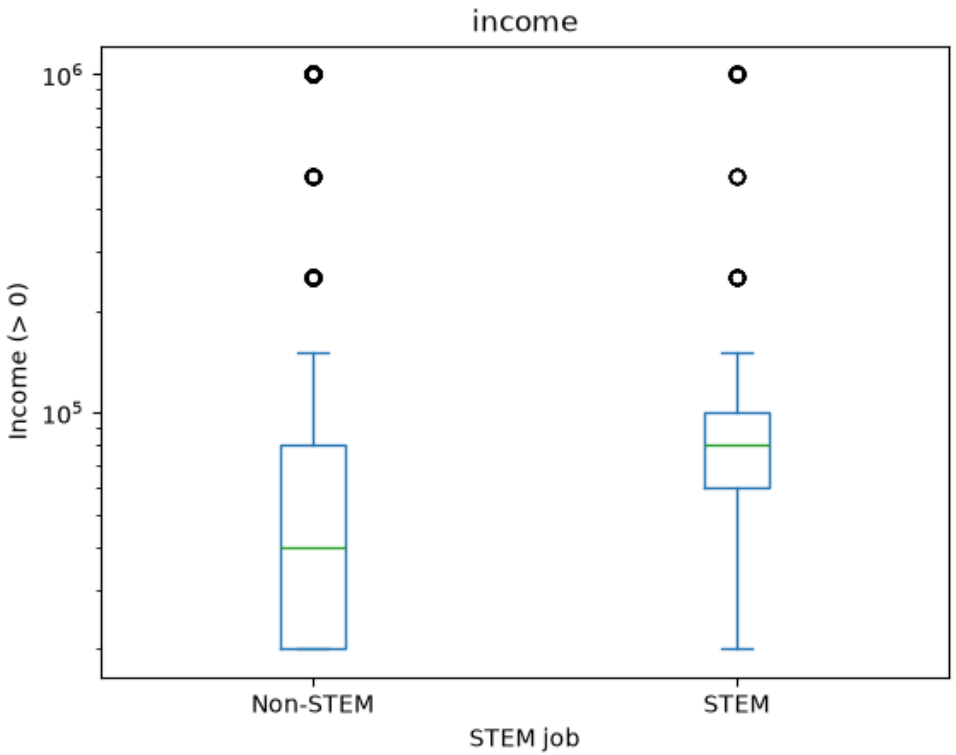
Context matters!

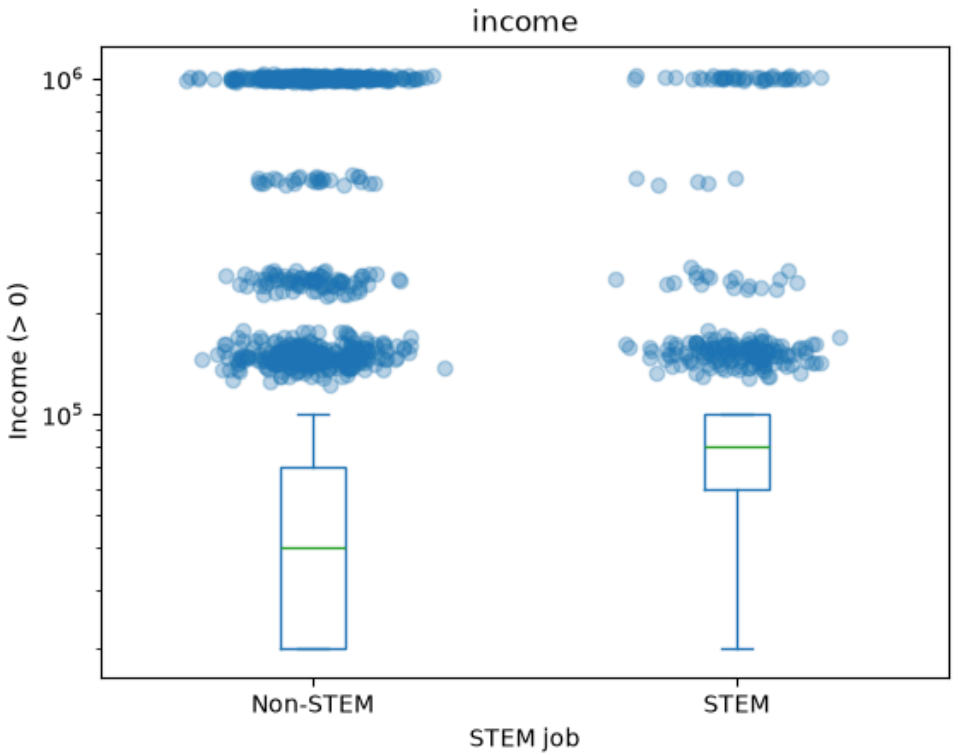
Look at the relationship between outliers and target (training dataset, of course)

> *What do the dots on box plots represent?*



Be careful with visual inspection





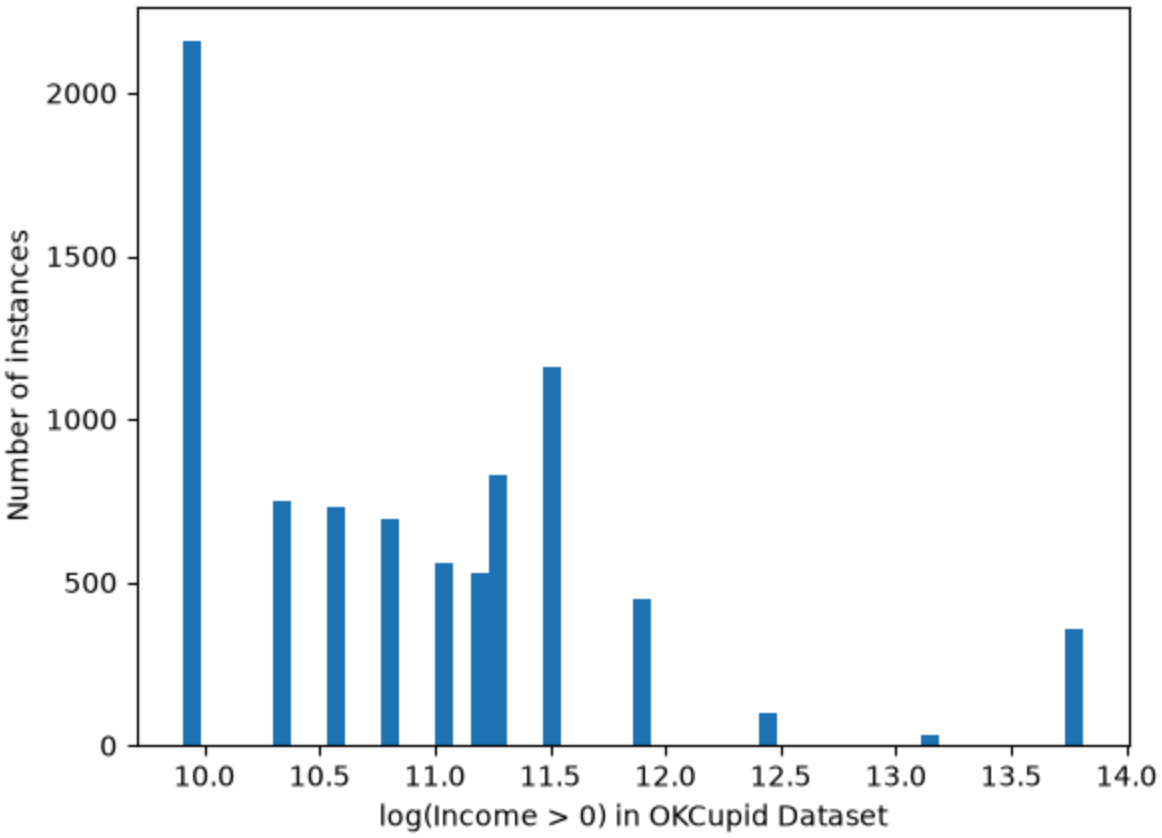
> *11% of samples are "outliers"!*

What to do about outliers?

As usual, very data- and model-dependent. Tree based methods are particularly impacted by outliers!

1. Data transformations
2. Drop the samples
3. Encode them somehow
4. Leave them alone

> *Any other ideas?*



Nonlinear transformations

- Transforming the data does not actually remove the outlier
- Can help make the relationship less extreme for linear models

Dropping the samples

- In general, not a thing I love to do
- If you drop a sample from training, you need to decide what to do at inference
- My opinion: Only do it if you're confident it's an error in the dataset

> *Can you think of an example we've seen so far?*

- What can you do at inference time when outliers are encountered?

Encoding outliers

A few other options that might fall into "encoding":

- Just like with missing values, binary column indicating outlier/inlier
- Remove the values and convert them to missing
- Bin or impose a cap (floor/ceiling) on the value
- Replace the numeric value with a rank or quantile bucket
- Probably other things!

Leave them alone

If your outliers are:

- Real values (not data entry or other errors)
- Representative of things that might happen during inference

Then you probably want to keep them!

> *Consider using a RobustScalar to standardize if you have lots of outliers*

Missing values and outliers in the target

- If you have missing values in the target, you probably want to avoid imputing
- This is a good case for dropping samples from **training**!
- Outliers are trickier — again, check if they're real or mistakes
- There may be a case for transforming the target, but be careful interpreting results

> *What does it mean if* $MSE(\mathbf{y} - \hat{\mathbf{y}}) > MSE(\log(\mathbf{y}) - \log(\hat{\mathbf{y}}))$*?*

Revisiting unsupervised learning

Learning patterns without labels, such as:

- Anomaly detection
- Density estimation
- Dimensionality reduction
- **Clustering**

> *Why is unsupervised learning important?*

Note: this content was originally in lecture 2

Clustering: k-means

- Goal: Partition N samples into k clusters
- Naive algorithm:
  i. Initialize k centroid locations
  ii. Assign each sample to the nearest centroid
  iii. Recalculate centroids for each cluster
  iv. Repeat steps 2-3 until clusters stabilize

$$\sum_{i=0}^{N} \min_{\mu_j \in C} (\|x_i - \mu_j\|^2)$$

k-means pros and cons

+ Intuitive
+ Easy to implement
+ Relatively scalable ($O(N)$)

- Need to know how many clusters
- Sensitive to initialization
- Sensitive to outliers
- Not guaranteed to converge to global minimum
- Doesn't like anisotropy and unequal variance

> *Scikit-learn has a great reference table of other clustering methods*

Coming up next

- Lecture: Fancy encoding and transformations
- Lab: Exploring missing and weird data
- Assignment 2